

MS-1 Python Solutions

Pandas + NumPy Code

Q1 — Pandas: Solar Farm Maintenance Analysis

Complete Pandas solution from the provided MS-1 Python PDF.

```
import pandas as pd

class SolarMaintenanceAnalyzer:

    def create_inspections_df(self, inspection_data):
        df = pd.DataFrame(
            inspection_data,
            columns=[
                "InspectionID",
                "SiteID",
                "Region",
                "InspectionDate",
                "OutputMWh",
                "DowntimeHours",
                "MaintenanceStatus"
            ]
        )
        return df

    def clean_inspection_data(self, df):
        df = df.dropna()

        df = df[
            (df["OutputMWh"] > 0) &
            (df["DowntimeHours"] >= 0) &
            (df["MaintenanceStatus"].isin(["Completed", "Scheduled"]))
        ]

        df = df.reset_index(drop=True)

        return df

    def add_attention_flag(self, df, downtime_threshold):
        df = df.copy()

        df["NeedsAttention"] = (
            df["DowntimeHours"] > downtime_threshold
        ).astype(int)

        return df

    def site_performance_summary(self, df):
        df = df.groupby("SiteID").agg(
            InspectionCount=("SiteID", "count"),
            TotalOutputMWh=("OutputMWh", "sum"),
            AverageDowntime=("DowntimeHours", "mean")
        ).reset_index()

        df["AverageDowntime"] = df["AverageDowntime"].round(2)

        df = df.sort_values("SiteID").reset_index(drop=True)

        return df

    def low_output_sites(self, df, output_threshold):
        df = df.groupby("SiteID")["OutputMWh"].sum().reset_index()

        df = df[df["OutputMWh"] < output_threshold]

        df = df.rename(
            columns={"OutputMWh": "TotalOutputMWh"}
        )

        df = df.sort_values("SiteID").reset_index(drop=True)
```

```
        return df

def regional_maintenance_cost(self, df):
    df = df[
        df["MaintenanceStatus"] == "Completed"
    ].copy()

    df["MaintenanceCost"] = (
        df["DowntimeHours"] * 250
    )

    df = df.groupby("Region")["MaintenanceCost"].sum().reset_index()

    df = df.sort_values("Region").reset_index(drop=True)

    return df
```

Q2 — NumPy: Cold-Storage Temperature Analyzer

Complete NumPy solution from the provided MS-1 Python PDF.

```
import numpy as np

def create_temperature_array(values):
    arr = np.array(values, dtype=np.float64)

    result = arr

    return result

def validate_temperature_array(arr):
    if arr.size == 0:
        result = False

    elif not np.issubdtype(arr.dtype, np.number):
        result = False

    elif np.all((arr >= -30.0) & (arr <= 10.0)):
        result = True

    else:
        result = False

    return result

def compute_temperature_stats(arr):
    mean = round(np.mean(arr), 2)

    std = round(np.std(arr), 2)

    maximum = round(np.max(arr), 2)

    minimum = round(np.min(arr), 2)

    result = (
        float(mean),
        float(std),
        float(maximum),
        float(minimum)
    )

    return result

def categorize_temperatures(arr):
    result = np.empty(arr.shape, dtype="<U7")

    result[
        (arr >= -30.0) & (arr <= -18.0)
    ] = "Frozen"

    result[
        (arr > -18.0) & (arr <= 5.0)
    ] = "Chilled"

    result[
        (arr > 5.0) & (arr <= 10.0)
    ] = "Warning"

    result[
        (arr < -30.0) | (arr > 10.0)
    ] = "Invalid"

    return result

def longest_warning_streak(arr):
    current = 0
```

```
longest = 0
for value in arr:
    if value > 5.0 and value <= 10.0:
        current = current + 1
        if current > longest:
            longest = current
    else:
        current = 0
result = longest
return result
```